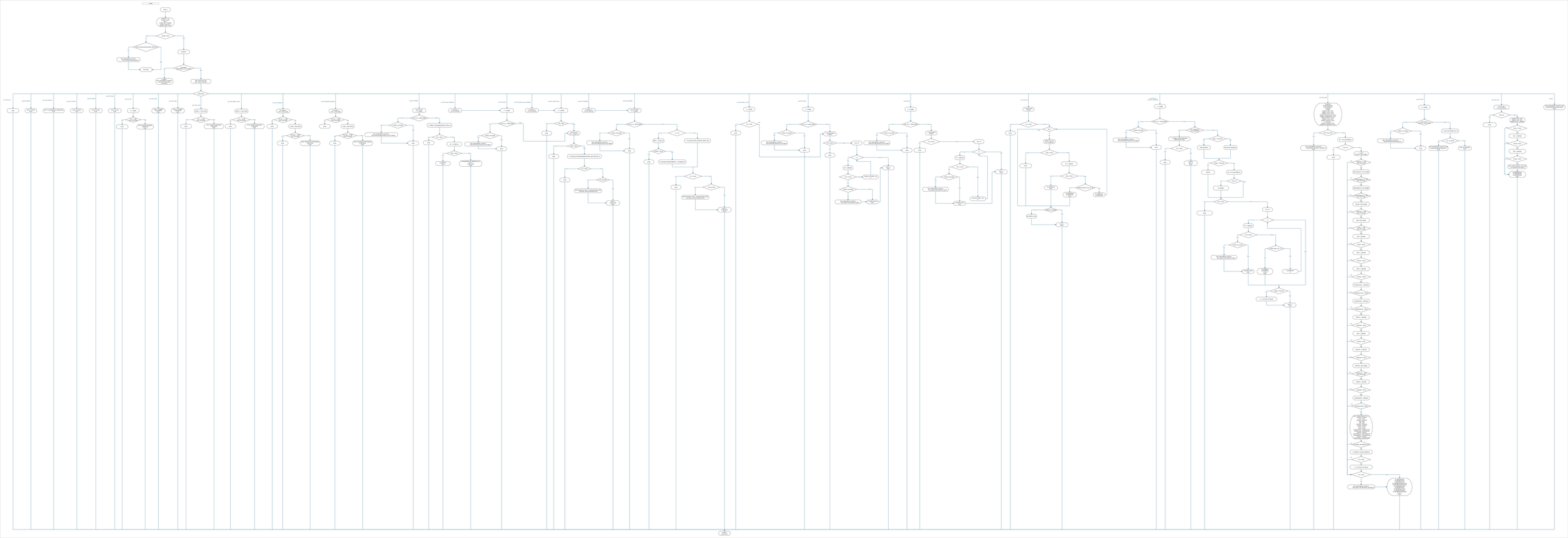

r_object
RFILE *p;
PyObject *v, *v2;
Py_ssize_t idx = 0;
long i, n;
int type, code = r_byte(p);
int flag, is_interned = 0;
PyObject *retval = NULL;
if (code == EOF)
if (PyErr_ExceptionMatches(PyExc_EOFError))
PyErr_SetString(PyExc_EOFError,
"EOF read where object expected");
return NULL;
p->depth++;
if (p->depth > MAX_MARSHAL_STACK_DEPTH)
p->depth--;
PyErr_SetString(PyExc_ValueError, "recursion limit exceeded");
return NULL;
flag = code & FLAG_REF;
type = code & ~FLAG_REF;
switch (type)
case TYPE_NULL:
break;
case TYPE_NONE:
retval = Py_None;
break;
case TYPE_STOPITER:
retval = Py_NewRef(PyExc_StopIteration);
break;
case TYPE_ELLIPSIS:
retval = Py_Ellipsis;
break;
case TYPE_FALSE:
retval = Py_False;
break;
case TYPE_TRUE:
retval = Py_True;
break;
case TYPE_INT:
n = r_long(p);
if (n == -1 && PyErr_Occurred())
break;
retval = PyLong_FromLong(n);
R_REF(retval);
break;
case TYPE_INT64:
retval = r_long64(p);
R_REF(retval);
break;
case TYPE_LONG:
retval = r_PyLong(p);
R_REF(retval);
break;
case TYPE_FLOAT:
double x = r_float_str(p);
if (x == -1.0 && PyErr_Occurred())
break;
retval = PyFloat_FromDouble(x);
R_REF(retval);
break;
case TYPE_BINARY_FLOAT:
double x = r_float_bin(p);
if (x == -1.0 && PyErr_Occurred())
break;
retval = PyFloat_FromDouble(x);
R_REF(retval);
break;
case TYPE_COMPLEX:
Py_complex c;
c.real = r_float_str(p);
if (c.real == -1.0 && PyErr_Occurred())
break;
c.imag = r_float_str(p);
if (c.imag == -1.0 && PyErr_Occurred())
break;
retval = PyComplex_FromCComplex(c);
R_REF(retval);
break;
case TYPE_BINARY_COMPLEX:
Py_complex c;
c.real = r_float_bin(p);
if (c.real == -1.0 && PyErr_Occurred())
break;
c.imag = r_float_bin(p);
if (c.imag == -1.0 && PyErr_Occurred())
break;
retval = PyComplex_FromCComplex(c);
R_REF(retval);
break;
case TYPE_STRING:
const char *ptr;
n = r_long(p);
if (n < 0 || n > SIZE32_MAX)
if (!PyErr_Occurred())
PyErr_SetString(PyExc_ValueError,
"bad marshal data (bytes object size out of range)");
break;
v = PyBytes_FromStringAndSize((char *)NULL, n);
if (v == NULL)
break;
ptr = r_string(n, p);
if (ptr == NULL)
Py_DECREF(v);
break;
memcpy(PyBytes_AS_STRING(v), ptr, n);
retval = v;
R_REF(retval);
break;
case TYPE_ASCII_INTERNED:
is_interned = 1;
_Py_FALLTHROUGH;
case TYPE_ASCII:
n = r_long(p);
if (n < 0 || n > SIZE32_MAX)
if (!PyErr_Occurred())
PyErr_SetString(PyExc_ValueError,
"bad marshal data (string size out of range)");
break;
case TYPE_SHORT_ASCII_INTERNED:
is_interned = 1;
_Py_FALLTHROUGH;
case TYPE_SHORT_ASCII:
n = r_byte(p);
if (n == EOF)
break;
const char *ptr;
ptr = r_string(n, p);
if (ptr == NULL)
break;
v = PyUnicode_FromKindAndData(PyUnicode_1BYTE_KIND, ptr, n);
if (v == NULL)
break;
if (is_interned)
PyInterpreterState *interp = _PyInterpreterState_GET();
_PyUnicode_InternImmortal(interp, &v);
retval = v;
R_REF(retval);
break;
case TYPE_INTERNED:
is_interned = 1;
_Py_FALLTHROUGH;
case TYPE_UNICODE:
const char *buffer;
n = r_long(p);
if (n < 0 || n > SIZE32_MAX)
if (!PyErr_Occurred())
PyErr_SetString(PyExc_ValueError,
"bad marshal data (string size out of range)");
break;
if (n != 0)
buffer = r_string(n, p);
if (buffer == NULL)
break;
v = PyUnicode_DecodeUTF8(buffer, n, "surrogatepass");
v = Py_GetConstant(Py_CONSTANT_EMPTY_STR);
if (v == NULL)
break;
if (is_interned)
PyInterpreterState *interp = _PyInterpreterState_GET();
_PyUnicode_InternImmortal(interp, &v);
retval = v;
R_REF(retval);
break;
case TYPE_SMALL_TUPLE:
n = r_byte(p);
if (n == EOF)
break;
case TYPE_TUPLE:
n = r_long(p);
if (n < 0 || n > SIZE32_MAX)
if (!PyErr_Occurred())
PyErr_SetString(PyExc_ValueError,
"bad marshal data (tuple size out of range)");
break;
v = PyTuple_New(n);
R_REF(v);
if (v == NULL)
break;
for (i = 0;
i < n
v2 = r_object(p);
if (v2 == NULL)
if (!PyErr_Occurred())
PyErr_SetString(PyExc_TypeError,
"NULL object in marshal data for tuple");
Py_SETREF(v, NULL);
break;
PyTuple_SET_ITEM(v, i, v2);
i++
retval = v;
break;
case TYPE_LIST:
n = r_long(p);
if (n < 0 || n > SIZE32_MAX)
if (!PyErr_Occurred())
PyErr_SetString(PyExc_ValueError,
"bad marshal data (list size out of range)");
break;
v = PyList_New(n);
R_REF(v);
if (v == NULL)
break;
for (i = 0;
i < n
v2 = r_object(p);
if (v2 == NULL)
if (!PyErr_Occurred())
PyErr_SetString(PyExc_TypeError,
"NULL object in marshal data for list");
Py_SETREF(v, NULL);
break;
PyList_SET_ITEM(v, i, v2);
i++
retval = v;
break;
case TYPE_DICT:
v = PyDict_New();
R_REF(v);
if (v == NULL)
break;
for (;;)
PyObject *key, *val;
key = r_object(p);
if (key == NULL)
break;
val = r_object(p);
if (val == NULL)
Py_DECREF(key);
break;
if (PyDict_SetItem(v, key, val) < 0)
Py_DECREF(key);
Py_DECREF(val);
break;
Py_DECREF(key);
Py_DECREF(val);
if (PyErr_Occurred())
Py_SETREF(v, NULL);
retval = v;
break;
case TYPE_SET:
case TYPE_FROZENSET:
n = r_long(p);
if (n < 0 || n > SIZE32_MAX)
if (!PyErr_Occurred())
PyErr_SetString(PyExc_ValueError,
"bad marshal data (set size out of range)");
break;
if (n == 0 && type == TYPE_FROZENSET)
v = _PyObject_CallNoArgs((PyObject *)&PyFrozenSet_Type);
if (v == NULL)
break;
R_REF(v);
retval = v;
break;
(type == TYPE_SET) ?
PySet_New(NULL)
PyFrozenSet_New(NULL);
if (type == TYPE_SET)
R_REF(v);
idx = r_ref_reserve(flag, p);
if (idx < 0)
Py_CLEAR(v);
if (v == NULL)
break;
for (i = 0;
i < n
v2 = r_object(p);
if (v2 == NULL)
if (!PyErr_Occurred())
PyErr_SetString(PyExc_TypeError,
"NULL object in marshal data for set");
Py_SETREF(v, NULL);
break;
if (PySet_Add(v, v2) == -1)
Py_DECREF(v);
Py_DECREF(v2);
v = NULL;
break;
Py_DECREF(v2);
i++
if (type != TYPE_SET)
v = r_ref_insert(v, idx, flag, p);
retval = v;
break;
case TYPE_CODE:
int argcount;
int posonlyargcount;
int kwonlyargcount;
int stacksize;
int flags;
PyObject *code = NULL;
PyObject *consts = NULL;
PyObject *names = NULL;
PyObject *localsplusnames = NULL;
PyObject *localspluskinds = NULL;
PyObject *filename = NULL;
PyObject *name = NULL;
PyObject *qualname = NULL;
int firstlineno;
PyObject *linetable = NULL;
PyObject *exceptiontable = NULL;
if (!p->allow_code)
PyErr_SetString(PyExc_ValueError,
"unmarshalling code objects is disallowed");
break;
idx = r_ref_reserve(flag, p);
if (idx < 0)
break;
v = NULL;
argcount = (int)r_long(p);
if (argcount == -1 && PyErr_Occurred())
posonlyargcount = (int)r_long(p);
if (posonlyargcount == -1 && PyErr_Occurred())
kwonlyargcount = (int)r_long(p);
if (kwonlyargcount == -1 && PyErr_Occurred())
stacksize = (int)r_long(p);
if (stacksize == -1 && PyErr_Occurred())
flags = (int)r_long(p);
if (flags == -1 && PyErr_Occurred())
code = r_object(p);
if (code == NULL)
consts = r_object(p);
if (consts == NULL)
names = r_object(p);
if (names == NULL)
localsplusnames = r_object(p);
if (localsplusnames == NULL)
localspluskinds = r_object(p);
if (localspluskinds == NULL)
filename = r_object(p);
if (filename == NULL)
name = r_object(p);
if (name == NULL)
qualname = r_object(p);
if (qualname == NULL)
firstlineno = (int)r_long(p);
if (firstlineno == -1 && PyErr_Occurred())
linetable = r_object(p);
if (linetable == NULL)
exceptiontable = r_object(p);
if (exceptiontable == NULL)
struct _PyCodeConstructor con = {
.filename = filename,
.name = name,
.qualname = qualname,
.flags = flags,
.code = code,
.firstlineno = firstlineno,
.linetable = linetable,
.consts = consts,
.names = names,
.localsplusnames = localsplusnames,
.localspluskinds = localspluskinds,
.argcount = argcount,
.posonlyargcount = posonlyargcount,
.kwonlyargcount = kwonlyargcount,
.stacksize = stacksize,
.exceptiontable = exceptiontable};
if (_PyCode_Validate(&con) < 0)
v = (PyObject *)_PyCode_New(&con);
if (v == NULL)
v = r_ref_insert(v, idx, flag, p);
if (v == NULL)
PyErr_SetString(PyExc_TypeError,
"NULL object in marshal data for code object");
Py_XDECREF(code);
Py_XDECREF(consts);
Py_XDECREF(names);
Py_XDECREF(localsplusnames);
Py_XDECREF(localspluskinds);
Py_XDECREF(filename);
Py_XDECREF(name);
Py_XDECREF(qualname);
Py_XDECREF(linetable);
Py_XDECREF(exceptiontable);
retval = v;
break;
case TYPE_REF:
n = r_long(p);
if (n < 0 || n >= PyList_GET_SIZE(p->refs))
if (!PyErr_Occurred())
PyErr_SetString(PyExc_ValueError,
"bad marshal data (invalid reference)");
break;
v = PyList_GET_ITEM(p->refs, n);
if (v == Py_None)
PyErr_SetString(PyExc_ValueError, "bad marshal data (invalid reference)");
break;
retval = Py_NewRef(v);
break;
case TYPE_SLICE:
Py_ssize_t idx = r_ref_reserve(flag, p);
if (idx < 0)
break;
PyObject *stop = NULL;
PyObject *step = NULL;
PyObject *start = r_object(p);
if (start == NULL)
stop = r_object(p);
if (stop == NULL)
step = r_object(p);
if (step == NULL)
retval = PySlice_New(start, stop, step);
r_ref_insert(retval, idx, flag, p);
Py_XDECREF(start);
Py_XDECREF(stop);
Py_XDECREF(step);
break;
default:
PyErr_SetString(PyExc_ValueError, "bad marshal data (unknown type code)");
break;
p->depth--;
return retval;